

Title : Improving Detection of Visually Challenging Automotive Components: An Empirical Evaluation of YOLO Training Strategies

Table of Contents

<i>Introduction</i>	<i>2</i>
1.2 Research Problem and Objectives.....	2
1.3 Research Questions	3
1.4 Key Challenges	3
1.5 Specific Contribution of the Dissertation	4
1.6 Contribution	4
1.7 Dissertation Structure	4
<i>Chapter 2: Foundations or Background</i>	<i>5</i>
2.1 Convolutional neural network	5
2.2 Object Detection Approaches	5
2.3 YOLO Architecture	6
2.4 Transfer Learning.....	7
2.5 Data Augmentation	8
<i>YOLO Architecture diagram</i>	9
<i>Chapter 3: Related Work</i>	<i>10</i>
3.1 Object Detection in Industrial Settings	10
3.2 Automotive Component Detection.....	10
3.3 YOLO for Domain-Specific Detection	12
3.4 Handling Class Difficulty and Imbalance	13
3.5 Research Gap.....	13
Chapter 4: Methodology :-.....	14
<i>References</i>	<i>24</i>

Introduction

The task addressed in this dissertation is object detection, where the input is an image of an automotive component and the output is a bounding box with a class label identifying the part. In computer vision, modern technologies are applicable to the problem of findings such as Defect detection, Quality inspections, Automated Inventory management and Logistics, Component identifications and tracking in assembly lines (Zhao et al., 2019). Previously, these problems were solved by using Convolution Neural Networks and its models (LeCun, Bengio and Hinton, 2015) but nowadays, parts may vary different in a size, colors, resolutions, brightness and its appearance.

Here, In the automotive sectors like Logistics and Workshops has various types of components of cars and machines. For human to find a different type of components is complex and time-consuming tasks. This problem can be solved by the latest AI models like YOLO (Redmon et al., 2016) and its reliable detection and localization of individual car parts and has become essential in workshops, insurance claims processing, spare-parts logistics and smart factories.

In this dissertation, automotive parts vary significantly in size, shape, orientation, texture and visual appearance and they often appear in cluttered backgrounds or under varying lighting conditions. It causes some parts detection **problem** and low accuracy result and we will improve and rectify this problem.

1.2 Research Problem and Objectives

Computer vision technologies are increasingly used in industrial and logistics environments to support tasks such as product identification. In deep learning, particularly CNN have significantly improved object detection performance (LeCun, Bengio and Hinton, 2015), but modern detection architectures such as YOLO you only look once allow models to identify and in real time (Redmon et al., 2016). Their effectiveness for detecting automotive parts under varying visual conditions requires systematic investigation.



1.3 Research Questions

- RQ1: Which automotive component categories are most challenging to detect, and what visual characteristics contribute to their difficulty?
- RQ2: How do model architecture, input resolution, and transfer learning affect detection accuracy for visually challenging components?
- RQ3: Which data augmentation strategies most effectively improve detection of hard-to-detect automotive parts?

This Problem matters in different fields and it occurs practical consequences like unreliable system in real workshops, higher error rates in logistics, limited adoption of automated visual recognitions also generates problem in insurance claiming companies. And it generates the gap and this dissertation sites the most existing report only aggregates metrics or focus on every single-component.

1.4 Key Challenges

In the automotive dataset, the study uses the public Roboflow 50-class automotive dataset with 8739 images, it evaluates Yolov8 and YOLO26 family models and latest models through six controlled experiments and it focusses on per class analysis rather designing a new architecture.

1.5 Specific Contribution of the Dissertation

- In the key points, here pre-class failure analysis that identifies which automotive parts are hardest to detect
- why systematic empirical comparison of YOLO training strategies like augmentation, transfer learning, resolutions how they help us to find the parts.
- Practical, Reproducible configuration recommendations for multi-class automotive detection.
- Fully documented open-sources implementation that allows other researchers and practitioners to reproduce the experiments.

1.6 Contribution

In the dissertation, Access to implementation for the practical solution, the complete training pipeline, experiment logs, results and source code are publicly available in my GitHub account repository. In the repository select the GPU and configure the environment and try to run the experiments and all the results are uploaded in Google drive. Here is the link: <https://github.com/manan36chauhan/automotive-component-detection>

1.7 Dissertation Structure

The remainder of the dissertation is organized as follows chapter 2 provides the foundational concepts and background, covering the CNN object detection approaches, transfer learning, data augmentation, and evaluation metrics. Next Chapter 3 presents a critical review of the related work in industrial object and strategies to handle the class difficulty. Moreover, in chapter 4 details the experiments methodology dataset, model configurations, six experiments, evaluation, implementation details and ethical considerations. In Chapter 5 reports the evaluation results and pre class analysis across all experiments. Finally, Chapter 6 summarizes contribution, limitation and direction for future work. References and appendices follow.

Chapter 2: Foundations or Background

2.1 Convolutional neural network

Convolutional Neural Network is the class of deep learning model particularly designed to process structured and grid like data, most commonly images (LeCun, Bengio and Hinton, 2015). Previously traditional neural networks, CNN exploit spatial hierarchies by learning local patterns like edges, textures and shapes which are then combined into high-level representations. This makes them highly effective for computer vision tasks like image classification, object detection, and segmentation. CNN use shared weights and local receptive fields, significantly reducing the number of parameters compared to fully connected networks thereby improving computational efficiency and generalization.

The main part of the Convolutional Network that included convolutional layers, pooling layers, activation functions and connected layers. Convolutional layers apply learnable filters kernels that slide over the input to extract the feature maps. Pooling layer reduces the spatial dimensions, typically via max and average pooling to lower computational cost and it can control overfitting. Activation function such as Rectified Linear Unit ReLU introduced non-linearity, to model the complex patterns. Finally this connected layers integrate extracted features and perform the final classification and prediction.

2.2 Object Detection Approaches

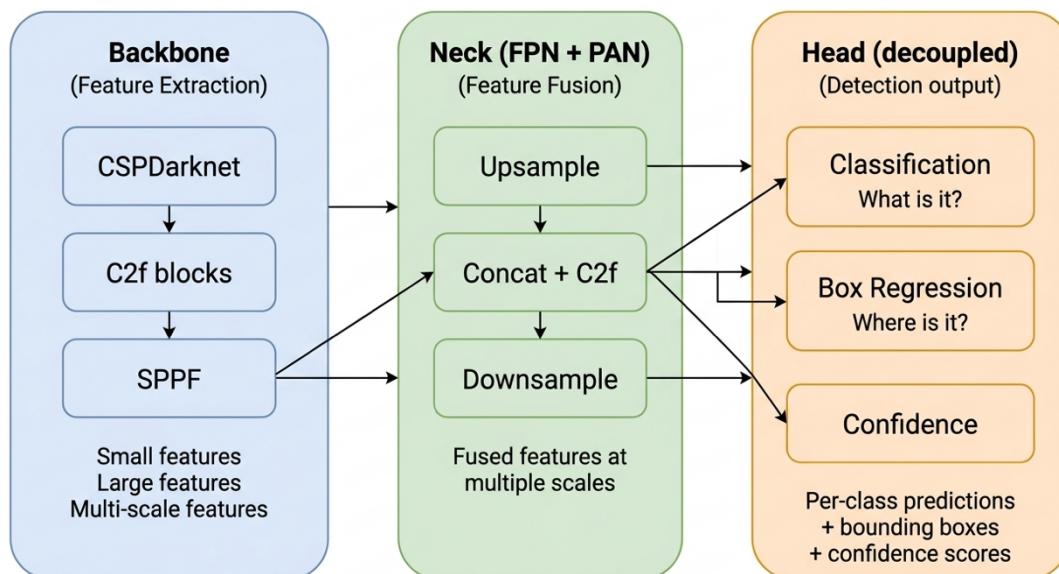
Object detection targets to both localization and classification of objects within an image, typically by predicting bounding boxes and corresponding class labels. In modern approaches fall into two categories for two stage detectors and one stage detectors.

Two-stage detectors such as Faster RCNN follows a sequential pipeline (Ren et al., 2017). In the first stage, Region proposal network RPN scans the image and generates candidate object regions. In the second phase these proposals are refined and categorized into object classes. This decoupling allows for high localization accuracy with strong performance on small or big overlapping objects. However, the multiple steps process that introduces computational overhead, making inference slower and less suitable for time application.

In contrast, YOLO you only look once (Redmon et al., 2016) and the Single shot multibox detector (Liu et al., 2016) performance detection in a single forward pass. They directly predict **bounding box and class probability from features** maps without an explicit proposal. This design significantly improves the speed and it enables real time performance at a slight cost in accuracy and especially for small objects and recent advancements have narrowed this gap.

2.3 YOLO Architecture

The YOLO architecture described an inserted image into grids and, for each grid cell it predicts a fixed number of bounding boxes including confidence scores and class probabilities. Modern YOLO variants were use of a backbone (CSPDarknet) for feature extraction, a neck (PANet or FPN) for multi-scale feature fusion (Lin et al., 2017), and a head for final predictions. Main innovations include anchor boxes anchor-free detection, and improvements in loss functions, such as Complete IoU (CIoU) loss.



YOLOv8 / YOLO26 architecture: single-pass detection from input image to bounding box predictions
Figure 1: Comprehensive overview of the YOLOv8/YOLO26 architecture, highlighting the feature extraction backbone, multi-scale fusion neck, and decoupled detection head.

The YOLO reflects continuous improvements in speed, accuracy, and flexibility:

- YOLOv1 :- This model real-time detection but it struggled with localization and small objects (Redmon et al., 2016).

- YOLOv2 :- (YOLO9000) This model improved the accuracy with batch normalization and anchor boxes(Redmon and Farhadi, 2017).
- YOLOv3 :- this version adopted multi-scale predictions and a deeper backbone (Redmon and Farhadi, 2018).
- YOLOv4 :- this model integrated an advanced training tricks (e.g., mosaic augmentation, CSPNet) (Bochkovskiy, Wang and Liao, 2020).
- YOLOv5:- this models from Ultralytics they emphasized usability, modularity, and PyTorch implementation(Jocher et al., 2023).
- YOLOv6–v8:- This model produced anchor-free designs, better loss functions, and optimized architectures for edge deployment.
- YOLOv26: introduces NMS-free end-to-end inference, Small-Target-Aware Label Assignment (STAL), and the MuSGD optimizer (Sapkota et al., 2025).

Overall, one stage detectors dominate in real-time applications, while two-stage detectors remain relevant for tasks requiring and maximum accuracy.

2.4 Transfer Learning

It is a Deep learning technique where a model trained on a large, general dataset is adapted to a specific task and domain. Instead of training a model from scratch which requires significant data and computational power transfer learning pretrained weights to accelerate the convergence and improve, performance especially when labeled data is less.

Additionally, models are pretrained on large scale datasets such as COCO (Common Objects in Context) (Lin et al., 2014), These models provides the diverse categories effect and real-world details. These pretrained models learn generic visual features like edges, textures, and shapes in early layers, and more complex semantic patterns in deeper layers. These learned representations of decisions are transferable across process (Zhuang et al., 2021).

There are two primary strategies for applying transfer learning:

1. Feature extraction: The pretrained model is used as a fixed feature extractor. Only the final layers will replaced and trained on new dataset, while earlier layers remain Deactivated.

2. Fine-tuning: The entire model or selected layers is retrained on target dataset with a lower learning rate. This allows the model to adapt its features to domain-specific characteristics.

Fine-tuning is important for such tasks, such as medical imaging for industrial defect detection, where the data distribution differs from general datasets. For example, a model pretrained on COCO can be fine-tuned to detect sewer defects by adjusting the final classification layers and retraining on annotated defect images.

Transfer learning reduces training time, improves generalization, and enables the effective modeling even with relatively small datasets. It is a standard practice in modern deep learning workflows.

2.5 Data Augmentation

Data augmentation is a strategy used to artificially increase the diversity and size of a training dataset by applying different kind of transformations to existing data (Shorten and Khoshgoftaar, 2019). It focuses primary on to improve model generalization, reduce overfitting, and make models more efficient and robust to variations in real-world data. Augmentation techniques or strategies can be categorized into **geometric, photometric (color-based), and composite methods.**

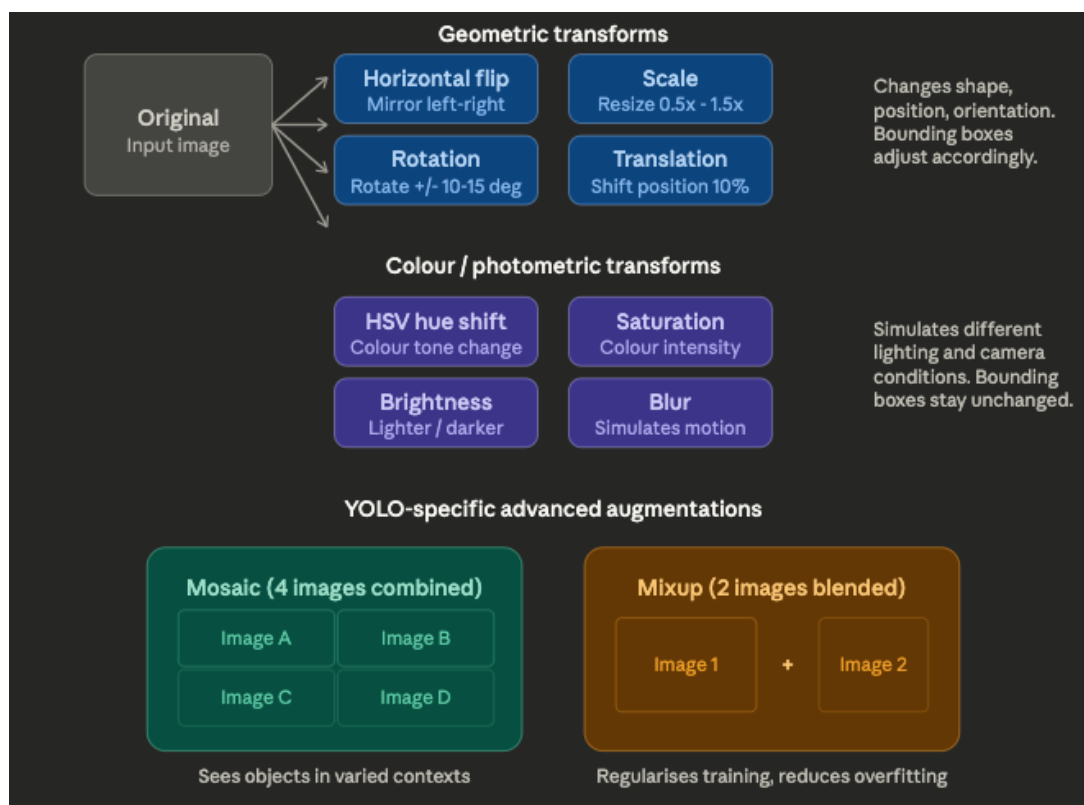
Geometric transformations allows the spatial structure of images. Common methods that enhance and modify including rotation, translation, scaling, flipping, and cropping of images or data. It help the model become invariant to object orientation and positioning. For object detection tasks, bounding boxes must be adjusted accordingly to the maintain alignment with transformed objects.

Color space augmentations modify or alter pixel intensity and color distributions. For example such as brightness, contrast, color saturation, or hue adjustments, as well as adding noise and blurring. It simulate different lighting conditions and sensor variations, improving robustness in diverse environments.

Advanced techniques include Mosaic augmentations, which combines four images into one during training which is given in YOLO(Bochkovski, Wang and Liao, 2020). This works with the model to multiple contexts and object scales in a sample, improving detection of small objects and enhancing contextual learning. Another method, MixUp, blends two images and

their labels by weighted average. This encourages smoother decision boundaries and reduces overfitting.

In modern object detection pipelines for YOLO models, augmentation is integrated into training loops and plays a critical role in performance. Properly configured augmentation can significantly improve the accuracy, particularly in datasets with the lower visually challenging limited samples and class imbalance.



YOLO Architecture diagram

Chapter 3: Related Work

3.1 Object Detection in Industrial Settings

Researchers have applied object detection widely in manufacturing for quality control to detecting surface defects or missing parts and in logistics for inventory tracking and automated handling. These systems mostly integrate with the robotics to support real-time decision making and. Industrial environments like on specific difficulties such as varying lighting, occlusions, cluttered backgrounds, and the need for low latency (Liu et al., 2023).

Early foundational work on deep convolutional architectures supported later adoption in industrial vision (Szegedy et al., 2015). It highlight the use of YOLO and Faster R-CNN in inspection tasks, with strong results under controlled conditions (Liu et al., 2023).

In contrast to the studies reviewed practically, which typically report only aggregate detection metrics, this dissertation thesis analyses each and every class performance across 50 automotive component categories this data set from Roboflow. It helps identify which specific components remain unreliable, undetectable and why information that aggregate metrics cannot reveal.

3.2 Automotive Component Detection

In Automotive Companies and Logistics As intelligent manufacturing and quality assurance systems have gained attraction to these solution, deep learning approaches based on the YOLO family models have seen wide adoption because they offer both accuracy and real time performance. In practice, Automotive applications typically it require identifying multiple parts, spotting defects such as cracks or misalignments, and confirming correct assembly within production lines.

Several recent research papers states that YOLO-based models work well for automotive defect detection. he has given a defect detection framework built on YOLOv7, designed specifically for identifying defects in automotive parts (Huang and Zhu., 2024). Their studies achieved high detection accuracy and proved robust at identifying surface level defects under controlled conditions. YOLOv7 strikes a useful balance between speed and accuracy for industrial inspection, though the evaluation mostly focuses on overall performance metrics.

There is limited discussion of how different defect types affect what the model can and cannot detect.

Previous research took a related approach, introducing an optimized YOLO-based model for car part detection with architectural modifications and training enhancements aimed at improving accuracy (Chandaluri et al., 2025). Their work targeted detection of multiple automotive parts in complex environments and to achieved better performance than baseline models. Modifications to the YOLO architecture, including improved feature extraction and multi scale detection, boosted detection capability. But the evaluation stays at an aggregate level. There is no detailed per class or each and every class analysis to show which components are harder to detect, or why its harder which we will demonstrate.

Real time automotive inspection has also been explored outside the lab. proposed an enhanced YOLO-based scanning portal system for vehicle inspection that allows real time detection of components as vehicles pass through inspection points(Ye et al., 2025). The system pairs object detection with hardware infrastructure to support automated logistics operations. (Okano et al., 2025) developed an automated dimensional inspection system that uses computer vision to measure and verify automotive components during production. Both system points to a uplifting the trend of embedding object detection into end to end industrial workflows where real time performance and reliability matter most.

Despite this progress several limitations persist. Most of the existing work focuses on either defect detection, recognition of a limited set of components, rather than comprehensive multi class detection across diverse automotive parts. The majority of studies report performance using aggregate metrics such as overall mAP, which can mask significant variations between classes. In real world scenarios, certain components, small parts, reflective surfaces, or visually similar structures, are inherently harder to detect. Yet there is limited investigation into why these classes remain difficult, even when sufficient training data is available. Few studies systematically evaluate whether training strategies such as data augmentation or resolution scaling actually help with these difficult classes.

YOLO-based approaches have shown strong results in automotive applications, but there is still a lack of detailed, class level analysis and systematic evaluation of detection performance. Filling these gaps is necessary for improving the reliability and robustness of object detection systems in real world automotive environments.

3.3 YOLO for Domain-Specific Detection

The YOLO (You Only Look Once) versions of models has become one of the most widely used object detection frameworks across domain specific applications, Its real time performance and competitive accuracy. In existing studies it is originally introduced YOLO by reframing object detection as a single regression problem, which allowed end to end prediction of bounding boxes and class probabilities in one forward pass (Redmon et al., 2016). then versions, including YOLOv5 to YOLOv8 (Jocher et al., 2023), they brought architectural improvements, better feature extraction, and improved training strategies that made the framework adaptable to a range of different domains.

YOLO-based models have been applied in every fields such as medical, agriculture, surveillance, and manufacturing. In medical imaging, YOLO has been used for tumour detection where high sensitivity is needed despite limited and imbalanced datasets. In agriculture it has been applied to crop and disease detection under difficult environmental conditions like varying illumination and occlusion. Surveillance systems utilize the YOLO variants for real time detection of pedestrians, vehicles, abnormal activities. the framework manages to handle specific constraints without sacrificing speed.

Training or Adapting YOLO for specific tasks usually involves modifying training pipelines and configurations rather than changing architecture at fundamental level. That include fine tuning pretrained models on specific datasets, adjusting resolution, and targeted augmentation to improve robustness. Some researchers also introduce architectural changes such as improved feature pyramid networks or attention mechanisms to improve detection of small and complex parts. Recent work in domains shows that adaptations can significantly improve overall detection performance, when paired with transfer learning and carefully curated datasets.

A recurring limitation across domain specific YOLO studies, though, is the reliance on aggregate performance metrics. Many works report high overall mAP values but do not break down performance by class to evaluate how the model handles challenging object categories. This matters in automotive contexts especially, where certain components may be visually similar, small in size, affected by reflections, occlusions. Without per class evaluation, it is hard to tell whether the performance improvements are spread across all classes or driven mostly by the easier ones. The existing literature, as result, offers limited insight into how YOLO models

actually perform on visually complex or underrepresented classes and what factors make those classes difficult.

YOLO has proven to be a powerful and flexible framework for domain specific object detection for out dissertation , but there is still a gap in understanding how it behaves and works at the class level. This is especially true in complex applications such as automotive component detection. Closing this gap is necessary for improving model reliability and ensuring consistent performance across all object categories.

This dissertation extends the analysis to the per-class level, examining how different YOLO configurations and augmentation techniques affect the detection of the most visually challenging automotive component categories.

3.4 Handling Class Difficulty and Imbalance

Performance variations in object detection often stem from both class imbalance and visual complexity. Even classes with many samples can underperform if objects are small, reflective, low-contrast, or visually similar to others. Automotive components frequently perform these traits against cluttered backgrounds like resolutions and visibility

Data augmentation increases sample and improves robustness (Shorten and Khoshgoftaar, 2019). Loss reweighting and specialised sampling methods addresses imbalance, but the empirical studies suggest augmentation often generates the more consistent gains for visually difficult classes (Kancharla et al., 2024).

This dissertation builds on these findings by testing whether targeted augmentation and resolution adjustments improve detection specifically for visually challenging classes identified through per-class analysis, rather than evaluating effects on aggregate metrics alone.

3.5 Research Gap

Existing work in industrial and automotive object detection has advanced the use of YOLO models for defect identification, component recognition, and real-time inspection. Many studies introduce architectural modifications, custom training enhancements, or system integrations and report strong overall results.

However, few important aspects remain unaddressed. No study has conducted a detailed or class by class failure or per-class failure analysis across a large multi-class

automotive dataset to identify which components are hardest to detect and why it happen. Few investigations distinguish whether visual complexity or data scarcity primarily explains underperformance. Systematic comparisons of standard training strategies such as different architectures, resolutions, augmentation policies, and transfer learning evaluated at the per-class level are also largely absent.

This dissertation fills these gaps through six controlled experiments on a 50-class automotive components dataset. All results receive evaluation at both aggregate and per-class levels, with particular attention to the visual characteristics that make certain parts difficult. The work provides actionable insights and reproducible resources to support more reliable visual systems in automotive and industrial applications.

Chapter 4: Methodology :-

4.1 Research Methodology

This research adopts an experimental and benchmarking approach to evaluate the object detection performance in the automotive domain. Rather than engineering an entirely new neural network, this study focuses on the systematic and empirical evaluation of existing training strategies. The methodological states aligns with accepted research practice within data science, which recognizes rigorous empirical evaluation as a critical mechanism for advance machine learning. The primary objective is to test how different hyperparameter configs, architectural variation, input resolution, augmentation strategies and transfer learning approaches specifically affect per-class detection performance, particularly for visually challenging automotive components.

A controlled experimental approach is highly appropriate because it enable the precise isolation of individuals independent variables such as models like architecture, input resolution, augmentation techniques, and weight initialization while holding all other parameters constants. This design allows performance changes to be reliably attributed to the strategy under investigation, thereby enhancing internal validity and minimising confounding factors. Similar benchmarking methodologies have been successfully employed in recent

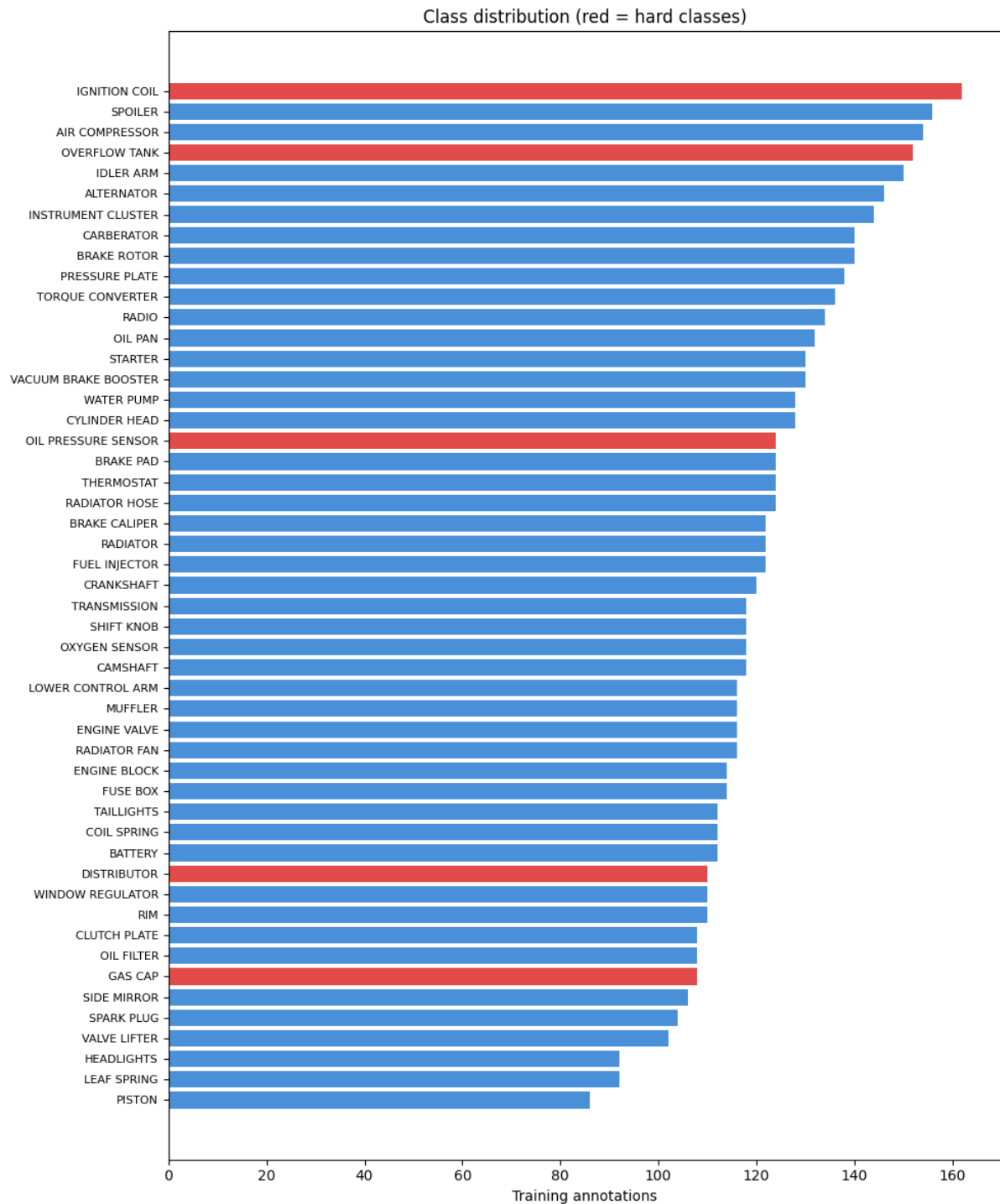
computer vision literature. For example, in 2024 Kancharla (2024) utilised a comparable experimental benchmarking framework to investigate the impacts of class imbalance on object detection models, demonstrating the value of controlled experiments in generating robust insights.

4.2 Dataset Description

The foundation of the experimental pipeline is the publicly available as car-parts-ybiev dataset and it is hosted on the Roboflow platform (Roboflow, n.d.). This dataset comprises 8,739 high-quality annotated images covering total 50 distinct classes for automotive components. Annotations are provided in the standard YOLO format with exactly one annotated object per image. This structure ensures seamless integration with the Ultralytics framework and it reduces the risk of overlapping and bounding box conflicts during training and evaluation.

The dataset is partitioned into 3 predefined subsets like a training set of 6,118 images, a validation set of 3,146 images, and a test set of 2,534 images. These splits were established by the dataset provider, and guaranteeing no data leakage between training, also validation, and testing phases and supporting a fair assessment of model generalisability.

Class distribution analysis in the training set reveals a relatively balanced dataset compared to many real world long tailed object detection benchmarks. The instances per class ranges from 86 samples of Piston to 162 samples of Ignition Coil, yielding an imbalance of approximately 1.9. This balance allows the study to focus on the effects of visual complexity rather than sample scarcity. The detailed distribution is illustrated in Figure .



The car-parts-ybiev dataset was selected for several reasons. It is one of the largest publicly available multi-class datasets dedicated to automotive component detection. The 50 classes offer sufficient visual diversity something like in texture, scale, shape, and appearance to observe the meaningful per-class performance variations in models. Finally, its public availability strongly supports the reproducibility, enabling other and researchers to replicate the same experiments and build upon the benchmarks.

4.3 Model Architecture and Configuration

The primary object detection framework used in this study is YOLOv8, developed by the Ultralytics (Jocher et al., 2023). YOLOv8 features an anchor free design with a detection head that separately processes objectness, classification, and regression tasks. 3 variants were evaluated to assess the impact of model capacity 1) YOLOv8n nano, 3.2 million parameters, 2) YOLOv8s small, 11.2 million parameters, and 3) YOLOv8m (medium, 25.9 million parameters). YOLOv8s serves as the primary baseline model due to its favourable balance between detection accuracy and computational efficiency, which is suited to the constraints of free-tier GPU resources.

For cross-generational comparison, YOLO26 (Ultralytics, 2026) was also evaluated. Released in early 2026, YOLO26 introduces several innovations, including NMS-free end-to-end inference, removal of Focal Loss, Small-Target-Aware Label Assignment (STAL), and the MuSGD optimizer. The YOLO26s variant was tested against YOLOv8s to determine whether these architectural advances improve detection of visually challenging automotive components.

Parameter	Value
Optimizer	SGD
Learning rate	0.01
Batch size	16 (8 for 800px)
Default image size	640px
Epochs	50 (80 for from-scratch)
Early stopping patience	10 (15 for from-scratch)
Random seed	42
Deterministic	True
Pretrained weights	COCO (unless stated)

Shared hyperparameter configuration across all experiments.

To ensure fair comparison across experiments, a shared hyperparameter configuration was applied unless a specific variable was deliberately modified.

4.4 Experiment Design

The methodology centres on six controlled experiments. Each isolates a single independent variable while keeping all other configurations constant. This systematic design enables clear attribution of performance changes to specific training strategies and directly maps to the research questions.

Exp	Name	Variable Tested	Values	RQ	Est. Time
1	Baseline	Reference point	YOLOv8s, 640px, COCO, default aug	RQ1	2 hours
2	Architecture	Model capacity	YOLOv8n / v8s / v8m	RQ2	3.30 hours
3	Resolution	Input size	320 / 640 / 800 px	RQ2	4.30 hours
4	Augmentation	Training diversity	None / Standard / Advanced	RQ3	5 hours
5	Transfer learning	Weight initialisation	COCO pretrained / From scratch	RQ2	4 hours
6	Cross-generational	Architecture generation	YOLOv8s vs YOLO26s	RQ2	132 min

Experiment 1 (Baseline) trains the primary YOLOv8s architecture utilising default settings. The purpose of this initial run is to establish a rigorous per-class performance baseline across all 50 classes and to formally identify which specific automotive components fall below the 0.85 mAP threshold. This foundational experiment directly answers RQ1.

Experiment 2 (Architecture) It compares performance of YOLOv8n with 3.2M params, YOLOv8s with 11.2M, and YOLOv8m with 25.9M. The objective is to test whether deploying larger models with more parameters provides the capacity necessary to learn better features for visually complex classes. Training times for this phase range from 67 to 3-4 hours depending on the model scale.

Experiment 3 (Resolution) tests the model across 320px, 640px, and 800px input resolutions. The underlying hypothesis is that higher resolution processing may reveal fine visual details required to distinguish highly similar-looking metallic components. To accommodate the larger 800px tensors in GPU memory, the batch size is explicitly reduced to 8.

Experiment 4 (Augmentation) compares three distinct augmentation regimes: no augmentation (all augmentation disabled), standard augmentation (geometric transforms including 10° rotation, 10% translation, 50% scale, 50% horizontal flip, alongside colour

transforms adjusting HSV hue 0.015, saturation 0.7, value 0.4), and advanced augmentation (standard techniques combined with mosaic 1.0 and mixup 0.15). A holistic approach is taken by comparing three augmentation levels. This reveals not just whether augmentation helps, but which combination of techniques is most effective for visually challenging classes.

Experiment 5 (Transfer Learning) contrasts a model initialised with COCO pretrained weights against a model initialised entirely with random weights (from scratch). Because a model trained from scratch requires a longer period to converge without pretrained baseline features, the training duration is extended to 80 epochs with an early stopping patience of 15.

Experiment 6 (YOLO26) conducts a cross-generational comparison between YOLOv8s and YOLO26s using identical experimental settings. YOLO26's STAL mechanism theoretically improves the detection of small or complex objects. This phase represents one of the first formal evaluations of YOLO26 on niche, domain-specific automotive data.

4.5 Evaluation Strategy

To comprehensively quantify model performance, a robust suite of standard computer vision metrics is employed, including Mean Average Precision at an Intersection over Union (IoU) threshold of 0.5 ($mAP@0.5$), Mean Average Precision across a range of thresholds ($mAP@0.5:0.95$), Precision, and Recall. The mathematical formulations and implications of these metrics have been thoroughly defined in Chapter 2.4 (Everingham et al., 2010; Padilla, Netto and da Silva, 2020).

The evaluation utilises a dual-level analytical strategy. First, all metrics are reported at an aggregate (overall) level to provide a macroscopic view. Second, a granular per-class analysis is conducted, focusing explicitly on the five hard classes identified during Experiment 1. A class is flagged as "hard" if its $mAP@0.5$ falls below 0.85. This strict threshold represents the point below which an automated detection system becomes fundamentally unreliable for practical deployment in real-world industrial environments.

Finally, Pearson and Spearman correlations are computed between the training sample count and the per-class mAP . This statistical correlation analysis definitively tests whether raw data quantity or the inherent visual complexity of the component better explains the observed performance variations.

4.6 Implementation Details

The computational requirements for this extensive pipeline necessitated a strategic hardware approach. Primary model training was conducted utilising the Kaggle cloud platform, capitalizing on their provision of dual NVIDIA T4 GPUs, each equipped with 15GB of VRAM, allowing for 30 hours of free compute time per week. Google Colab, utilising a single T4 GPU, served as the secondary training environment. Depending on the complexity of the architecture and resolution, training times ranged efficiently from 42.5 minutes (the shortest, being Experiment 3 at 320px) to 202.9 minutes (the longest, Experiment 2 utilising YOLOv8m). Total compute time across all experiments was approximately 25 hours.

The software environment was carefully standardised to ensure stability. The pipeline executed using Python, leveraging the PyTorch deep learning framework with CUDA acceleration. The Ultralytics provided native support for both YOLOv8 and YOLO26. The

Roboflow was used to automate dataset downloads, while Matplotlib and Seaborn handled visualisations. and SciPy was deployed for statistical analyses (Pearson and Spearman correlations), and OpenCV was utilised for image processing.

To guarantee reproducibility of the study, the seed was globally fixed to 42, and deterministic training algorithms were explicitly enabled. The entirety of the source code is publicly available and archived at the project's GitHub repository:

<https://github.com/manan36chauhan/automotive-component-detection>. All the experimental results serialised, while the trained model weights were securely archived on Google Drive. Furthermore, successful ONNX model exports were conducted to explain the deployment readiness.

4.7 Ethical Considerations

This study is strictly based on the secondary analysis of a publicly available dataset sourced from Roboflow, which exclusively contains imagery of inanimate automotive components. Consequently, no human participants are involved, and the dataset contains no personal or personally identifiable information (PII). Formal ethical approval was sought and successfully obtained from the dissertation supervisor prior to the commencement of the study. All frameworks, algorithms, and tools deployed in methodology are open-source. Finally, it is explicitly noted that the trained models intended for academic benchmarking and are not deployed in any safety-critical systems or autonomous vehicles.

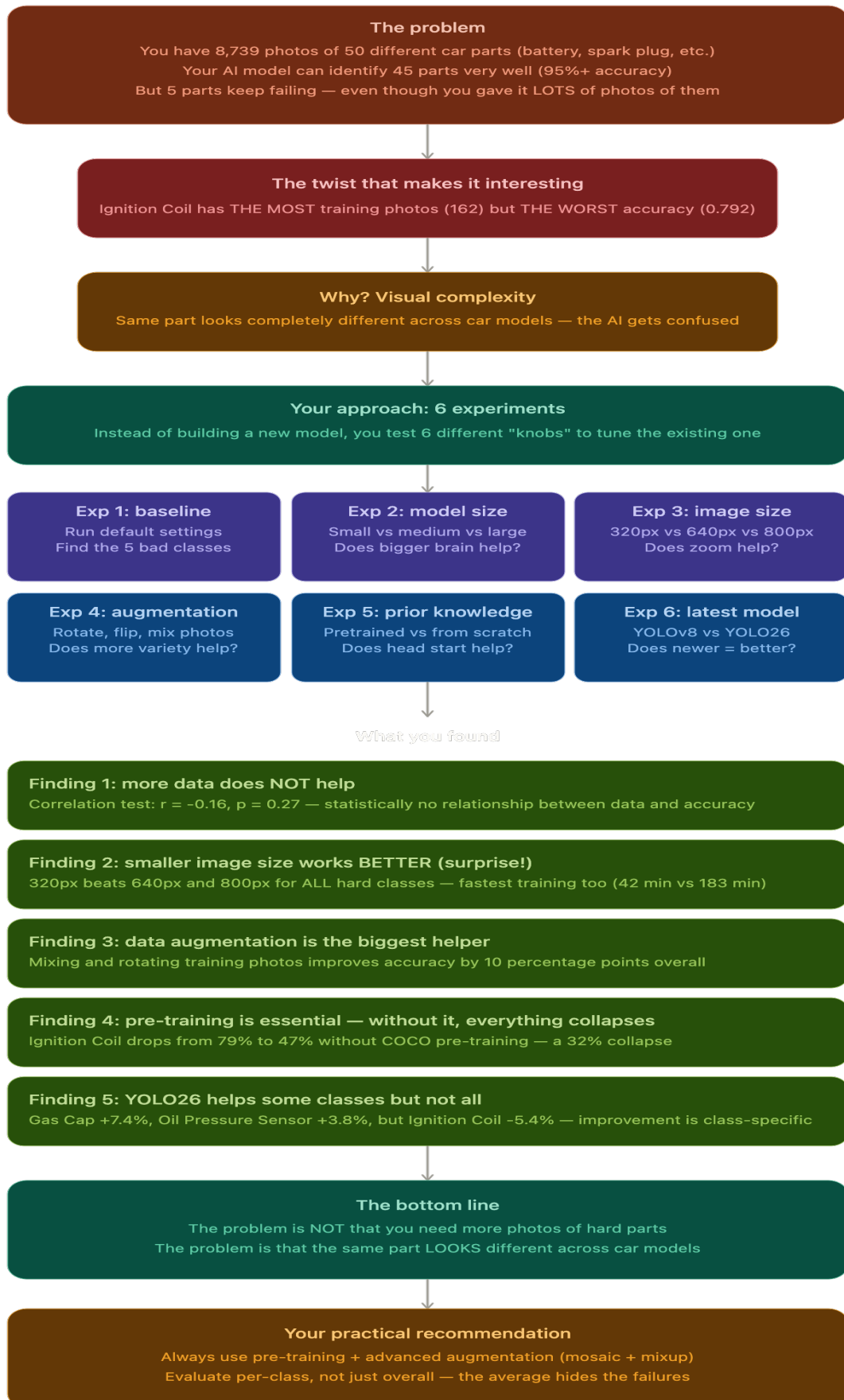
Extras:- Why these 5 classes are visually challenging

Why these 5 classes are visually challenging

Visual properties that explain low detection performance despite sufficient training data



Your project in plain English

Full code on GitHub: github.com/manan36chauhan/automotive-component-detection

Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2010) 'The Pascal Visual Object Classes (VOC) Challenge', *International Journal of Computer Vision*, 88(2), pp. 303–338. Available at: <https://doi.org/10.1007/s11263-009-0275-4>.

Jocher, G., Chaurasia, A. and Qiu, J. (2023) *Ultralytics YOLOv8*. GitHub repository. Available at: <https://github.com/ultralytics/ultralytics>

Kancharla, S.L. (2024) *Cost-Effective Re-Layouting of a Dynamic Manufacturing Facility with Collision-Free Material Handling*. Master's Thesis. Otto-von-Guericke-University Magdeburg.

Padilla, R., Netto, S.L. and da Silva, E.A.B. (2020) 'A Survey on Performance Metrics for Object-Detection Algorithms', in *Proceedings of the International Workshop on Signal Processing Systems (IWSSIP)*.

Roboflow (n.d.) *car-parts-ybiev dataset*. Available at: <https://universe.roboflow.com/team-data/car-parts-ybiev>

Ultralytics (2026) *YOLO26*. Ultralytics Documentation. Available at: <https://docs.ultralytics.com/models/yolo26/>

References

- Anupama, H.S., Ranjitha, R. and Srinivas, K. (2024) 'Comparative analysis of deep learning models for car part image segmentation', in Recent Trends in Computer Vision. Springer, pp. 147-162.
- Bochkovskiy, A., Wang, C.Y. and Liao, H.Y.M. (2020) 'YOLOv4: Optimal Speed and Accuracy of Object Detection', arXiv preprint, arXiv:2004.10934.
- Chandaluri, R. and Vasanthi, P. (2025) 'Optimized car parts detection with advanced feature fusion and attention modules', Scientific Reports.
- Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2010) 'The PASCAL Visual Object Classes (VOC) Challenge', International Journal of Computer Vision, 88(2), pp. 303-338.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep Residual Learning for Image Recognition', in Proceedings of the IEEE CVPR.
- Huang, H. and Zhu, K. (2024) 'Automotive Parts Defect Detection Based on YOLOv7', Electronics, 13(10), p. 1817.
- Jocher, G. et al. (2023) Ultralytics YOLOv8. Available at: <https://github.com/ultralytics/ultralytics> (Accessed: 28 March 2026).
- Kancharla, K.R. et al. (2024) 'Class Imbalance in Object Detection: An Experimental Diagnosis and Study of Mitigation Strategies', arXiv preprint, arXiv:2403.07113.
- Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) 'ImageNet Classification with Deep Convolutional Neural Networks', in Advances in Neural Information Processing Systems.
- LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', Nature, 521(7553), pp. 436-444.
- Lin, T.Y. et al. (2017) 'Feature Pyramid Networks for Object Detection', in Proceedings of the IEEE CVPR.
- Lin, T.Y. et al. (2014) 'Microsoft COCO: Common Objects in Context', in Proceedings of the ECCV.
- Liu, W. et al. (2016) 'SSD: Single Shot MultiBox Detector', in Proceedings of the ECCV.

- Liu, Z. et al. (2023) 'Deep learning methods for object detection in smart manufacturing: A survey', *Journal of Manufacturing Systems*.
- Padilla, R., Netto, S.L. and da Silva, E.A.B. (2020) 'A Survey on Performance Metrics for Object-Detection Algorithms', in *Proceedings of IWSSIP*.
- Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You Only Look Once: Unified, Real-Time Object Detection', in *Proceedings of the IEEE CVPR*, pp. 779-788.
- Redmon, J. and Farhadi, A. (2017) 'YOLO9000: Better, Faster, Stronger', in *Proceedings of the IEEE CVPR*.
- Redmon, J. and Farhadi, A. (2018) 'YOLOv3: An Incremental Improvement', *arXiv preprint*, arXiv:1804.02767.
- Ren, S., He, K., Girshick, R. and Sun, J. (2017) 'Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks', *IEEE TPAMI*, 39(6), pp. 1137-1149.
- Sapkota, R. et al. (2025) 'YOLO26: Key Architectural Enhancements and Performance Benchmarking', *arXiv preprint*, arXiv:2509.25164.
- Shorten, C. and Khoshgoftaar, T.M. (2019) 'A survey on Image Data Augmentation for Deep Learning', *Journal of Big Data*, 6(1), pp. 1-48.
- Szegedy, C. et al. (2015) 'Going deeper with convolutions', in *Proceedings of the IEEE CVPR*.
- Ye, X. et al. (2025) 'Enhanced YOLO and Scanning Portal System for Vehicle Component Detection', *PMC*.
- Zhao, Z., Zheng, P., Xu, S. and Wu, X. (2019) 'Object Detection with Deep Learning: A Review', *IEEE TNNLS*, 30(11), pp. 3212-3232.
- Zhuang, F. et al. (2021) 'A Comprehensive Survey on Transfer Learning', *Proceedings of the IEEE*, 109(1), pp. 43-76.